

Name: J. Renuka Devi

Reg No: 192573007

Course Code: CSA0925RD

Course Name: Java programming

CO4- AT2 - Lab Practice

Java Lab Practice – Applets, AWT Components and Layout Managers

Question 1 – Applet Life Cycle

Develop a Java Applet to demonstrate the **Applet Life Cycle** methods: init(), start(), paint(), stop(), and destroy().

Display a suitable message in the applet window whenever each life-cycle method is invoked. Observe the sequence of execution by running the applet and interacting with it.

Expected Outcome:

Students should understand the execution sequence and purpose of each Applet life-cycle method.

Applet Life Cycle:

```
import java.applet.Applet;
import java.awt.Graphics;

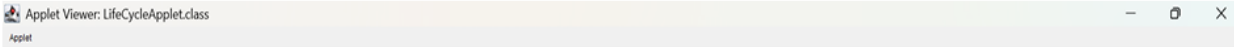
public class LifeCycleApplet extends Applet {
    String message = "";

    public void init() {
        message = "init() method invoked";
        System.out.println("init() called");
    }
}
```

```
}  
  
public void start() {  
    message = "start() method invoked";  
    System.out.println("start() called");  
}  
  
public void paint(Graphics g) {  
    g.drawString("Java Applet Life Cycle", 100, 80);  
    g.drawString(message, 100, 120);  
    System.out.println("paint() called");  
}  
  
public void stop() {  
    System.out.println("stop() called");  
}  
  
public void destroy() {  
    System.out.println("destroy() called");  
}  
}
```

HTML Code:

```
<html>  
<body>  
<applet code="LifeCycleApplet.class"  
    width="400"  
    height="200">  
</applet>  
</body>  
</html>
```



paint() method called

Applet started.

Question 2 – Generating Graphic Images Using Applet

Develop a Java Applet that uses the **Graphics class** to generate a simple graphical scene.

The applet should draw:

- A house
- Sun
- Trees
- Road
- Clouds

Use methods such as `drawLine()`, `drawRect()`, `drawOval()`, `fillRect()`, `fillOval()`, and appropriate colors.

Expected Outcome:

Students should demonstrate the use of the `Graphics class` for drawing shapes and images.

```
import java.applet.Applet;
```

```
import java.awt.*;

public class SceneApplet extends Applet {

    public void paint(Graphics g) {

        // Sky
        g.setColor(Color.cyan);
        g.fillRect(0, 0, 700, 400);

        // Sun
        g.setColor(Color.yellow);
        g.fillOval(550, 40, 80, 80);

        // Sun rays
        g.setColor(Color.orange);
        g.drawLine(590, 25, 590, 5);
        g.drawLine(590, 135, 590, 155);
        g.drawLine(535, 80, 515, 80);
        g.drawLine(645, 80, 665, 80);

        // Clouds
        g.setColor(Color.white);

        g.fillOval(70, 60, 80, 40);
        g.fillOval(110, 40, 100, 60);
        g.fillOval(180, 60, 80, 40);

        g.fillOval(400, 90, 80, 40);
        g.fillOval(440, 70, 100, 60);
        g.fillOval(510, 90, 80, 40);

        // Ground
        g.setColor(Color.green);
        g.fillRect(0, 270, 700, 130);

        // Road
        g.setColor(Color.gray);
        g.fillRect(0, 320, 700, 80);

        // Road markings
        g.setColor(Color.white);

        for (int x = 20; x < 700; x += 100) {
            g.fillRect(x, 355, 50, 5);
        }
    }
}
```

```

// House body
g.setColor(Color.orange);
g.fillRect(250, 170, 200, 120);

// House outline
g.setColor(Color.black);
g.drawRect(250, 170, 200, 120);

// Roof
g.setColor(Color.red);

g.drawLine(230, 170, 350, 80);
g.drawLine(350, 80, 470, 170);
g.drawLine(230, 170, 470, 170);

// Door
g.setColor(Color.darkGray);
g.fillRect(325, 220, 50, 70);

g.setColor(Color.black);
g.drawRect(325, 220, 50, 70);

// Windows
g.setColor(Color.white);

g.fillRect(275, 200, 45, 40);
g.fillRect(380, 200, 45, 40);

g.setColor(Color.black);

g.drawRect(275, 200, 45, 40);
g.drawRect(380, 200, 45, 40);

// Trees
drawTree(g, 80, 190);
drawTree(g, 520, 190);

// Extra oval outline
g.setColor(Color.green);
g.drawOval(70, 170, 70, 70);
}

void drawTree(Graphics g, int x, int y) {

// Trunk

```

```
g.setColor(new Color(139, 69, 19));
g.fillRect(x, y + 50, 35, 90);

// Leaves
g.setColor(Color.green);
g.fillOval(x - 25, y, 85, 80);
}
```

HTML Code:

```
<html>

<body>

<applet code="SceneApplet.class"

    width="700"

    height="400">

</applet>

</body>
</html>
```



Question 3 – Calculator Application Using Applet Parameters

Develop a **Calculator Applet** that accepts two numbers and an arithmetic operator through **HTML/Applet parameters**.

The applet should perform:

- Addition
- Subtraction
- Multiplication
- Division

Use the `getParameter()` method to retrieve the values passed to the applet.

Expected Outcome:

Students should understand how parameters are passed from an HTML page to an Applet and processed in Java.

Java Code:

```
import java.applet.Applet;
import java.awt.Graphics;

public class CalculatorApplet extends Applet {

    double num1;
    double num2;
    double result;

    String operator;

    public void init() {

        num1 = Double.parseDouble(
            getParameter("num1")
        );

        num2 = Double.parseDouble(
            getParameter("num2")
        );

        operator = getParameter("operator");

        if (operator.equals("+")) {

            result = num1 + num2;

        } else if (operator.equals("-")) {
```

```

        result = num1 - num2;

    } else if (operator.equals("*")) {

        result = num1 * num2;

    } else if (operator.equals("/")) {

        if (num2 != 0) {
            result = num1 / num2;
        }
    }
}

public void paint(Graphics g) {

    g.drawString(
        "Calculator Applet",
        100,
        60
    );

    g.drawString(
        num1 + " " +
        operator + " " +
        num2 + " = " +
        result,
        100,
        100
    );
}
}

```

HTML Code:

```

<html>

<body>

<applet code="CalculatorApplet.class"
    width="400"
    height="200">

<param name="num1" value="20">

<param name="num2" value="10">

```



```
<param name="operator" value="+">
```

```
</applet>
```

```
</body>
```

```
</html>
```



Applet Viewer: CalculatorApplet.class

Applet

CALCULATOR APPLET

Number 1: 20.0

Number 2: 10.0

Operator: +

Result: 30.0

Question 4 – Student Profile Using AWT Components

Develop a **Student Profile Application** using Java AWT components such as:

- Label
- TextField
- Checkbox
- CheckboxGroup
- Choice
- TextArea
- Button
- List

The application should accept student details such as:

- Name
- Register Number

- Gender
- Department
- Year
- Skills/Programming Languages

When the **Generate Report** button is clicked, display the entered student information in a List or TextArea.

Expected Outcome:

Students should demonstrate the use of multiple AWT components and event handling.

Student Profile Using AWT Components:

```
import java.awt.*;
import java.awt.event.*;

public class StudentProfile extends Frame implements ActionListener {

    Label nameLabel, regLabel, genderLabel;
    Label deptLabel, yearLabel, skillsLabel;
    TextField nameField, regField;

    Checkbox male, female;
    CheckboxGroup genderGroup;
    Choice department, year;
    List skills;
    TextArea result;
    Button generate;

    StudentProfile() {
        setTitle("Student Profile");
        setSize(600, 600);
        setLayout(new FlowLayout());
        nameLabel = new Label("Name:");
        nameField = new TextField(25);
        regLabel = new Label("Register Number:");
        regField = new TextField(20);
```

```
genderLabel = new Label("Gender:");
genderGroup = new CheckboxGroup();
male = new Checkbox("Male", genderGroup, false);
female = new Checkbox("Female", genderGroup, false);
deptLabel = new Label("Department:");
department = new Choice();
department.add("CSE");
department.add("ECE");
department.add("EEE");
department.add("IT");
yearLabel = new Label("Year:");
year = new Choice();
year.add("I Year");
year.add("II Year");
year.add("III Year");
year.add("IV Year");
skillsLabel = new Label("Skills:");
skills = new List(5, true);
skills.add("Java");
skills.add("Python");
skills.add("C");
skills.add("C++");
skills.add("SQL");
generate = new Button("Generate Report");
result = new TextArea(10, 50);
add(nameLabel);
add(nameField);
add(regLabel);
```

```

add(regField);
add(genderLabel);
add(male);
add(female);
add(deptLabel);
add(department);
add(yearLabel);
add(year);
add(skillsLabel);
add(skills);
add(generate);
add(result);

generate.addActionListener(this);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    String selectedSkills = "";
    String[] selected = skills.getSelectedItems();
    for (String skill : selected) {
        selectedSkills += skill + " ";
    }
}

```

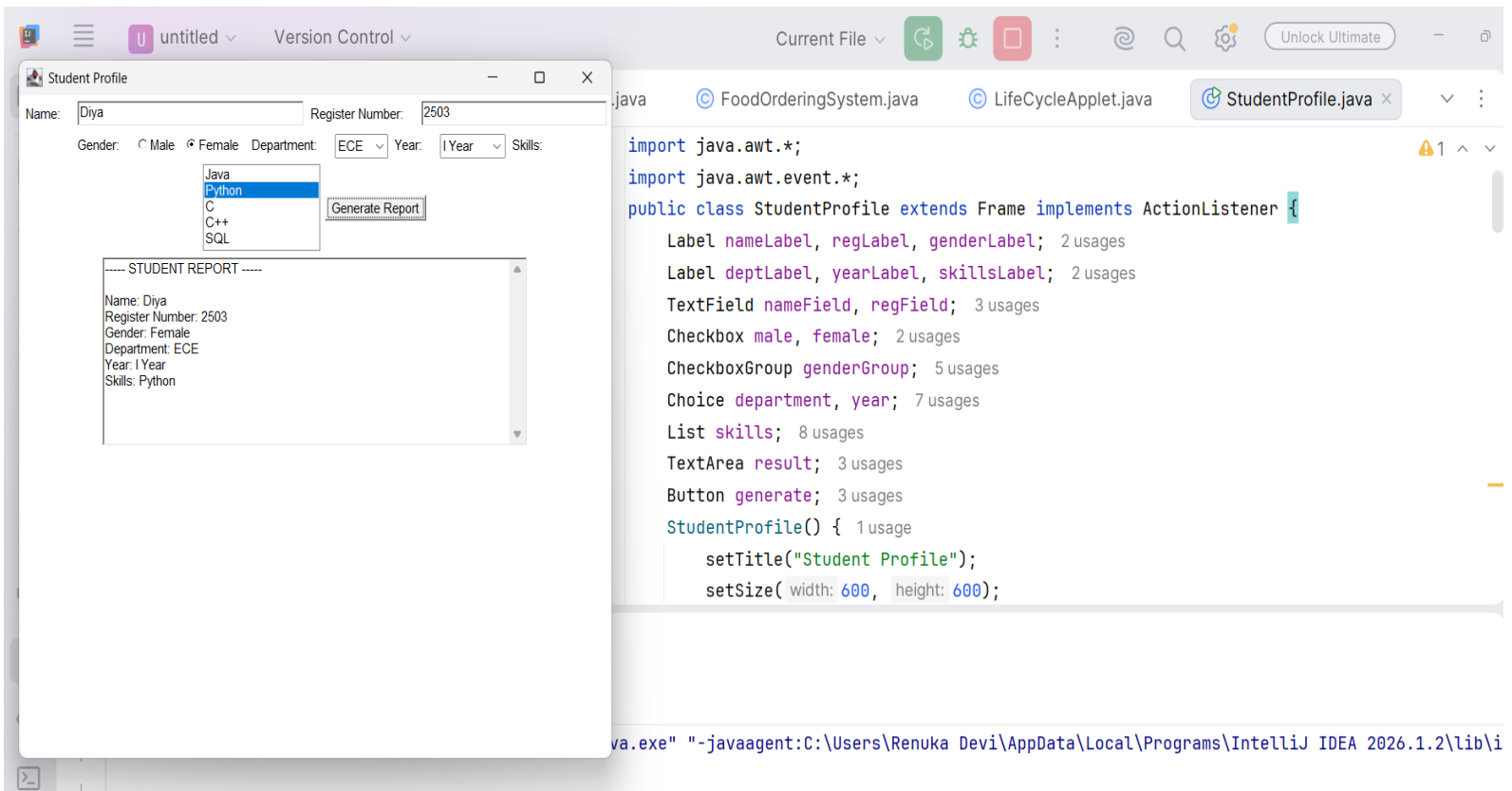
```

String gender = "Not Selected";
if (genderGroup.getSelectedCheckbox() != null) {
    gender =
        genderGroup.getSelectedCheckbox().getLabel();
}
result.setText(
    "----- STUDENT REPORT -----\n\n" +
    "Name: " + nameField.getText() + "\n" +
    "Register Number: " + regField.getText() + "\n" +
    "Gender: " + gender + "\n" +
    "Department: " +
        department.getSelectedItem() + "\n" +
    "Year: " +
        year.getSelectedItem() + "\n" +
    "Skills: " + selectedSkills
);
}

public static void main(String[] args) {
    new StudentProfile();
}

```

}



Question 5 – GridLayout with Event Handling

Develop a Java AWT application using **GridLayout** to create a simple form or calculator interface.

Arrange the components in a grid and implement appropriate **event handling** using ActionListener.

For example, create a calculator with buttons:

```
7 8 9 +
4 5 6 -
1 2 3 *
0 = C /
```

Perform the corresponding operation when a button is clicked.

Expected Outcome:

Students should understand GridLayout and button-based event handling.

GridLayout with Event Handling

```
import java.awt.*;
```

```

import java.awt.event.*;

public class GridCalculator extends Frame
    implements ActionListener {

    TextField display;

    double num1, num2;
    String operator = "";

    GridCalculator() {

        setTitle("GridLayout Calculator");
        setSize(400, 500);

        setLayout(new BorderLayout());

        display = new TextField();
        display.setEditable(false);

        add(display, BorderLayout.NORTH);

        Panel panel = new Panel();

        panel.setLayout(new GridLayout(4, 4, 5, 5));

        String[] buttons = {
            "7", "8", "9", "+",
            "4", "5", "6", "-",
            "1", "2", "3", "*",
            "0", "=", "C", "/"
        };

        for (String text : buttons) {

            Button button = new Button(text);

            button.addActionListener(this);

            panel.add(button);
        }

        add(panel, BorderLayout.CENTER);

        addWindowListener(new WindowAdapter() {

```

```

        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    String command = e.getActionCommand();

    // Number button
    if (command.matches("[0-9]")) {

        display.setText(
            display.getText() + command
        );
    }

    // Clear
    else if (command.equals("C")) {

        display.setText("");
        operator = "";
    }

    // Equal
    else if (command.equals("=")) {

        if (display.getText().equals("")) {
            return;
        }

        num2 = Double.parseDouble(
            display.getText()
        );

        double result = 0;

        if (operator.equals("+")) {
            result = num1 + num2;
        }
        else if (operator.equals("-")) {
            result = num1 - num2;
        }
    }
}

```



```

        else if (operator.equals("*")) {
            result = num1 * num2;
        }
        else if (operator.equals("/")) {
            result = num1 / num2;
        }

        display.setText(String.valueOf(result));
    }

    // Operator
    else {

        if (!display.getText().equals("")) {

            num1 = Double.parseDouble(
                display.getText()
            );

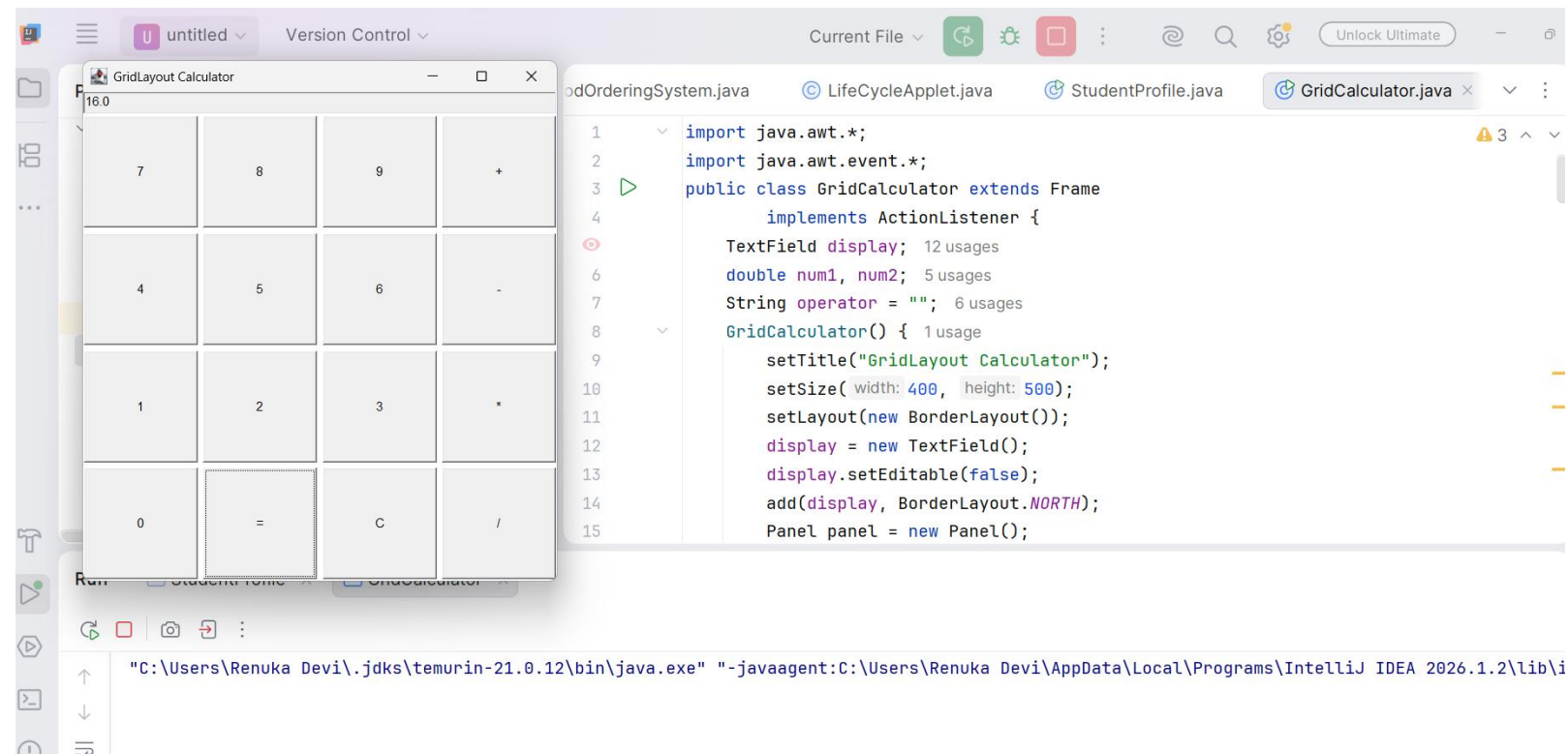
            operator = command;

            display.setText("");
        }
    }
}

public static void main(String[] args) {

    new GridCalculator();
}
}

```



Question 6 – GridBagLayout with Event Handling

Develop a Java AWT **Student Registration Form** using GridBagLayout.

The form should contain:

- Name
- Register Number
- Department
- Email
- Gender
- Year
- Submit Button
- Reset Button

Use GridBagConstraints to properly position and align the components.

Implement event handling for the **Submit** and **Reset** buttons.

Expected Outcome:

Students should understand flexible component positioning using GridBagLayout and event handling.

GridBagLayout with Event Handling

```
import java.awt.*;
import java.awt.event.*;

public class StudentRegistration extends Frame
    implements ActionListener {

    TextField nameField;
    TextField regField;
    TextField emailField;

    Choice department;
    Choice year;

    Checkbox male;
    Checkbox female;
    CheckboxGroup gender;

    Button submit;
    Button reset;

    GridBagConstraints gbc;

    StudentRegistration() {

        setTitle("Student Registration Form");
        setSize(550, 450);

        setLayout(new GridBagLayout());

        gbc = new GridBagConstraints();

        gbc.insets = new Insets(8, 8, 8, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        nameField = new TextField(20);
        regField = new TextField(20);
        emailField = new TextField(20);

        department = new Choice();
        department.add("CSE");
        department.add("ECE");
        department.add("EEE");
        department.add("IT");

        year = new Choice();
```

```
year.add("I Year");
year.add("II Year");
year.add("III Year");
year.add("IV Year");

gender = new CheckboxGroup();

male = new Checkbox(
    "Male", gender, false
);

female = new Checkbox(
    "Female", gender, false
);

submit = new Button("Submit");
reset = new Button("Reset");

addComponent(
    new Label("Name:"), 0, 0
);

addComponent(
    nameField, 1, 0
);

addComponent(
    new Label("Register Number:"), 0, 1
);

addComponent(
    regField, 1, 1
);

addComponent(
    new Label("Department:"), 0, 2
);

addComponent(
    department, 1, 2
);

addComponent(
    new Label("Email:"), 0, 3
);
```

```

addComponent(
    emailField, 1, 3
);

addComponent(
    new Label("Gender:"), 0, 4
);

Panel genderPanel = new Panel();

genderPanel.add(male);
genderPanel.add(female);

addComponent(
    genderPanel, 1, 4
);

addComponent(
    new Label("Year:"), 0, 5
);

addComponent(
    year, 1, 5
);

Panel buttonPanel = new Panel();

buttonPanel.add(submit);
buttonPanel.add(reset);

addComponent(
    buttonPanel, 1, 6
);

submit.addActionListener(this);
reset.addActionListener(this);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

setVisible(true);
}

```

```

void addComponent(
    Component component,
    int x,
    int y) {

    gbc.gridx = x;
    gbc.gridy = y;

    add(component, gbc);
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == submit) {

        String genderValue = "Not Selected";

        if (gender.getSelectedCheckbox() != null) {

            genderValue =
                gender.getSelectedCheckbox()
                    .getLabel();
        }

        System.out.println(
            "----- REGISTRATION DETAILS -----"
        );

        System.out.println(
            "Name: " + nameField.getText()
        );

        System.out.println(
            "Register Number: "
            + regField.getText()
        );

        System.out.println(
            "Department: "
            + department.getSelectedItems()
        );

        System.out.println(
            "Email: "
            + emailField.getText()
        );
    }
}

```

```

    );

    System.out.println(
        "Gender: " + genderValue
    );

    System.out.println(
        "Year: " + year.getSelectedItem()
    );
}

else if (e.getSource() == reset) {

    nameField.setText("");
    regField.setText("");
    emailField.setText("");

    gender.setSelectedCheckbox(null);

    department.select(0);
    year.select(0);
}

}

public static void main(String[] args) {

    new StudentRegistration();
}

}

```

The screenshot displays an IDE environment with a Java Swing application window titled "Student Registration Form" and its source code in "StudentRegistration.java".

Student Registration Form Window:

- Name:
- Register Number:
- Department:
- Email:
- Gender: ☐ Male ☒ Female
- Year:
- Buttons:

StudentRegistration.java Code:

```

1  import java.awt.*;
2  import java.awt.event.*;
3  public class StudentRegistration extends Frame
4      implements ActionListener {
5      TextField nameField; 4 usages
6      TextField regField; 4 usages
7      TextField emailField; 4 usages
8      Choice department; 8 usages
9      Choice year; 8 usages
10     Checkbox male; 2 usages
11     Checkbox female; 2 usages
12     CheckboxGroup gender; 6 usages
13     Button submit; 4 usages
14     Button reset; 4 usages
15     GridBagConstraints gbc; 6 usages

```

Run Console Output:

```

register number: 192573007
Department: CSE
Email: reena@gmail.com
Gender: Female
Year: II Year

```

Question 7 – BorderLayout with Event Handling

Develop a Java AWT application using **BorderLayout**.

Divide the window into:

- NORTH – Header
- SOUTH – Buttons
- EAST – Options
- WEST – Menu
- CENTER – Display area

Add buttons in the interface and implement event handling so that clicking different buttons changes the content displayed in the center area.

Expected Outcome:

Students should understand the five regions of BorderLayout and event-driven programming.

BorderLayout with Event Handling

```
import java.awt.*;
import java.awt.event.*;

public class BorderApplication extends Frame
    implements ActionListener {

    Label display;

    BorderApplication() {

        setTitle("BorderLayout Application");
        setSize(700, 450);

        setLayout(
            new BorderLayout(10, 10)
        );

        // NORTH - Header
        Label header = new Label(
            "STUDENT MANAGEMENT SYSTEM",
            Label.CENTER
        );

        add(header, BorderLayout.NORTH);

        // SOUTH - Buttons
        Panel southPanel = new Panel();
```



```
Button home = new Button("Home");
Button report = new Button("Report");
Button exit = new Button("Exit");

southPanel.add(home);
southPanel.add(report);
southPanel.add(exit);

add(
    southPanel,
    BorderLayout.SOUTH
);

// WEST - Menu
Panel westPanel = new Panel(
    new GridLayout(3, 1)
);

Button students = new Button(
    "Students"
);

Button faculty = new Button(
    "Faculty"
);

Button courses = new Button(
    "Courses"
);

westPanel.add(students);
westPanel.add(faculty);
westPanel.add(courses);

add(
    westPanel,
    BorderLayout.WEST
);

// EAST - Options
Panel eastPanel = new Panel(
    new GridLayout(3, 1)
);

Button add = new Button("Add");
Button update = new Button("Update");
```

```

Button delete = new Button("Delete");

eastPanel.add(add);
eastPanel.add(update);
eastPanel.add(delete);

add(
    eastPanel,
    BorderLayout.EAST
);

// CENTER - Display
display = new Label(
    "Welcome to Student Management System",
    Label.CENTER
);

add(
    display,
    BorderLayout.CENTER
);

// Event handling
home.addActionListener(this);
report.addActionListener(this);
exit.addActionListener(this);

students.addActionListener(this);
faculty.addActionListener(this);
courses.addActionListener(this);

add.addActionListener(this);
update.addActionListener(this);
delete.addActionListener(this);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

setVisible(true);
}

public void actionPerformed(ActionEvent e) {

```

```

        String command =
            e.getActionCommand();

        if (command.equals("Exit")) {

            dispose();

        } else {

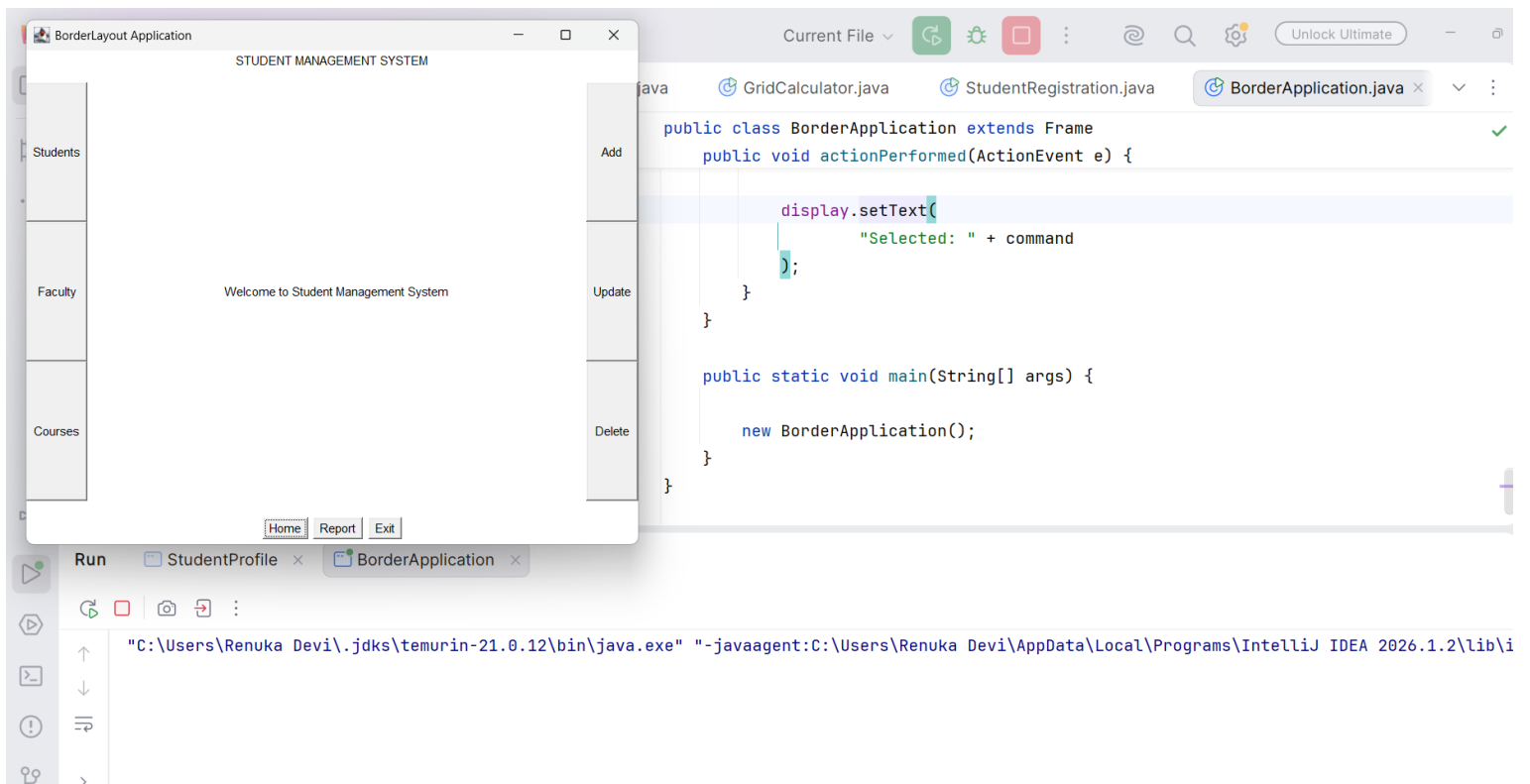
            display.setText(
                "Selected: " + command
            );
        }
    }

    public static void main(String[] args) {

        new BorderApplication();

    }
}

```



Question 8 – CardLayout with Event Handling

Develop a Java AWT application using **CardLayout** to create a multi-page interface.

Create at least three cards:

- **Card 1:** Student Information
- **Card 2:** Academic Information
- **Card 3:** Confirmation/Report

Provide **Next**, **Previous**, and **Submit** buttons to navigate between the cards.

Implement event handling to switch between cards using CardLayout.

Expected Outcome:

Students should understand how CardLayout can be used to create multiple screens within a single window.

CardLayout with Event Handling

```
import java.awt.*;
import java.awt.event.*;

public class CardApplication extends Frame
    implements ActionListener {

    CardLayout cardLayout;
    Panel cards;

    Button next;
    Button previous;
    Button submit;

    CardApplication() {

        setTitle("Student Registration");
        setSize(600, 400);

        // CardLayout
        cardLayout = new CardLayout();

        cards = new Panel(cardLayout);

        // -----
        // CARD 1
        // -----

        Panel card1 = new Panel(
            new FlowLayout()
```

```

);

card1.add(
    new Label("STUDENT INFORMATION")
);

card1.add(
    new Label("Name:")
);

card1.add(
    new TextField(20)
);

card1.add(
    new Label("Register Number:")
);

card1.add(
    new TextField(15)
);

// -----
// CARD 2
// -----

Panel card2 = new Panel(
    new FlowLayout()
);

card2.add(
    new Label("ACADEMIC INFORMATION")
);

card2.add(
    new Label("Department:")
);

Choice department =
    new Choice();

department.add("CSE");
department.add("ECE");
department.add("EEE");
department.add("IT");

```

```

card2.add(department);

card2.add(
    new Label("Year:")
);

Choice year =
    new Choice();

year.add("I Year");
year.add("II Year");
year.add("III Year");
year.add("IV Year");

card2.add(year);

// -----
// CARD 3
// -----

Panel card3 = new Panel(
    new FlowLayout()
);

card3.add(
    new Label("CONFIRMATION / REPORT")
);

TextArea report =
    new TextArea(
        "Student information submitted successfully.",
        6,
        35
    );

card3.add(report);

// Add cards
cards.add(card1, "Card1");
cards.add(card2, "Card2");
cards.add(card3, "Card3");

add(
    cards,
    BorderLayout.CENTER
);

```

```

// -----
// Navigation Buttons
// -----

Panel buttonPanel =
    new Panel();

previous =
    new Button("Previous");

next =
    new Button("Next");

submit =
    new Button("Submit");

buttonPanel.add(previous);
buttonPanel.add(next);
buttonPanel.add(submit);

add(
    buttonPanel,
    BorderLayout.SOUTH
);

// Event handling
previous.addActionListener(this);
next.addActionListener(this);
submit.addActionListener(this);

addWindowListener(
    new WindowAdapter() {

        public void windowClosing(
            WindowEvent e) {

            dispose();
        }
    }
);

setVisible(true);
}

public void actionPerformed(

```

```

       (ActionEvent e) {

String command =
    e.getActionCommand();

if (command.equals("Next")) {

    cardLayout.next(cards);
}

else if (command.equals("Previous")) {

    cardLayout.previous(cards);
}

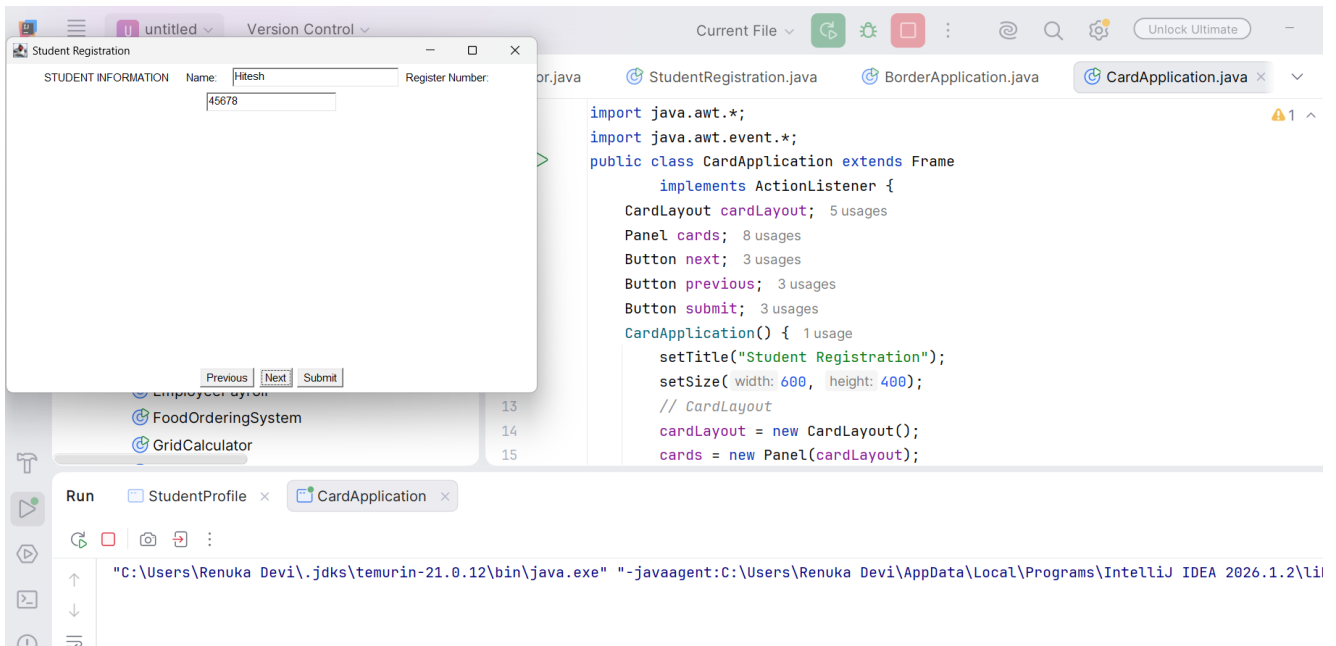
else if (command.equals("Submit")) {

    cardLayout.show(
        cards,
        "Card3"
    );
}
}

public static void main(String[] args) {

    new CardApplication();
}
}

```



Lab Practice Evaluation Focus

Students will be evaluated based on:

1. Correct implementation of the required Java concepts.
2. Proper use of AWT components.
3. Correct implementation of event handling.
4. Appropriate use of layout managers.
5. User-friendly interface design.
6. Successful compilation and execution.
7. Ability to explain the program and output during the lab examination.